

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

CSE 4802: Software Maintenance Lab

Lab 5
Code Clone Detection

Submitted By: Antara Arifa Mullick
Student ID: 210042132
Program: B.Sc. in Software Engineering
Department: Computer Science and Engineering

Task Author: Lutfun Nahar Lota

1 Introduction

SkillScribe is a web-based learning platform developed using React, Next.js, TypeScript, and TSX. The application contains separate features for learners, instructors, and administrators. Its modules include profile management, course browsing, a course player, community interaction, and administrative management components.

As the project contains many pages and reusable interface elements, similar code may appear in different files. In this lab, the project was analyzed to locate duplicated code and study how it may affect software quality and maintenance.

2 Tool Overview

jscpd was selected as the code clone detection tool. It scans source files and finds repeated token sequences that satisfy a chosen minimum number of lines and tokens. For every detected clone, it reports the two file locations, line ranges, duplicated lines, and duplicated tokens.

The tool was selected because it supports JavaScript, TypeScript, JSX, and TSX files and can analyze the complete SkillScribe project from the command line. It also produces console, HTML, and JSON reports. The HTML report was especially useful because it displayed the overall statistics, clone locations, and the detected code blocks.

3 Methodology

The following steps were followed:

1. The SkillScribe project was opened in Command Prompt from its root directory.
2. Node.js, npm, and jscpd were checked before running the analysis. The installed versions were Node.js 20.12.0, npm 10.5.0, and jscpd 5.0.14.
3. The project was scanned for JavaScript, JSX, TypeScript, and TSX files.
4. A minimum clone size of 5 lines and 50 tokens was used.
5. Dependency and generated folders such as `node_modules`, `.git`, `dist`, `build`, `coverage`, and `.next` were ignored.
6. Console, HTML, and JSON reporters were enabled.
7. The generated HTML report was opened to inspect clone locations and selected code blocks.

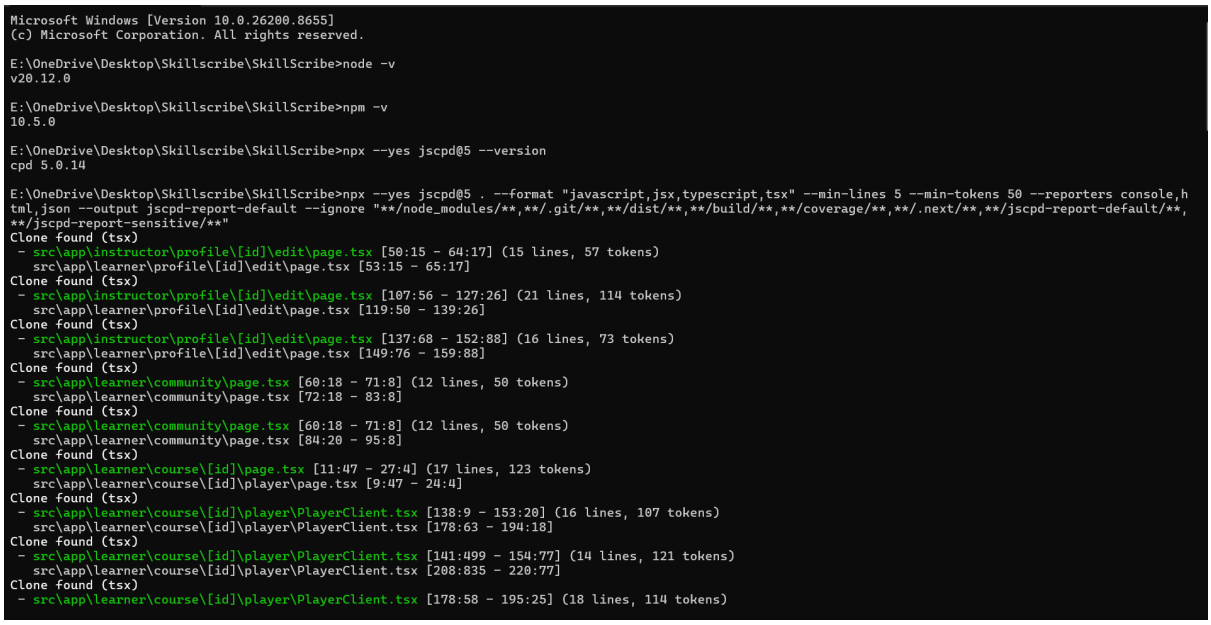


Figure 1: Tool versions, analysis command, and initial clone findings

4 Results

The analysis completed successfully and found **55 code clones** in the project. Most of the duplication occurred in TSX files.

Table 1: Summary of jscpd results

Format	Files	Lines	Clones	Duplicated Lines	Duplicated Tokens
JavaScript	1	18	0	0 (0.00%)	0 (0.00%)
TSX	226	22,835	54	775 (3.39%)	4,682 (3.72%)
TypeScript	25	11,449	1	14 (0.12%)	147 (0.30%)
Total	**252**	**34,302**	**55**	**789 (2.30%)**	**4,829 (2.75%)**

The overall duplicated-line percentage was **2.30%**. This is not a very high percentage for a project of this size, but the 55 detected clones show that several repeated structures can still be improved.

Format	Files analyzed	Total lines	Total tokens	Clones found	Duplicated lines	Duplicated tokens
javascript	1	18	52	0	0 (0.00%)	0 (0.00%)
tsx	226	22835	125885	54	775 (3.39%)	4682 (3.72%)
typescript	25	11449	49525	1	14 (0.12%)	147 (0.30%)
Total:	252	34302	175462	55	789 (2.30%)	4829 (2.75%)

```
Found 55 clones.
HTML report saved to jscpd-report-default\jscpd-report.html
JSON report saved to jscpd-report-default\jscpd-report.json
time: 6.26s

💡 Auto-refactor with AI: npx skills add https://github.com/kucherenko/jscpd --skill dry-refactoring
🚨 New: Gangsta Agents – discipline your AI coding → gangsta.page
💖 Support jscpd project → https://opencollective.com/jscpd

E:\OneDrive\Desktop\Skillscribe\SkillScribe>
```

Figure 2: Final result displayed in the terminal



Figure 3: HTML dashboard showing the overall clone statistics

The report listed clone locations from profile pages, community functions, course pages, course-player components, legal pages, and administrative components.

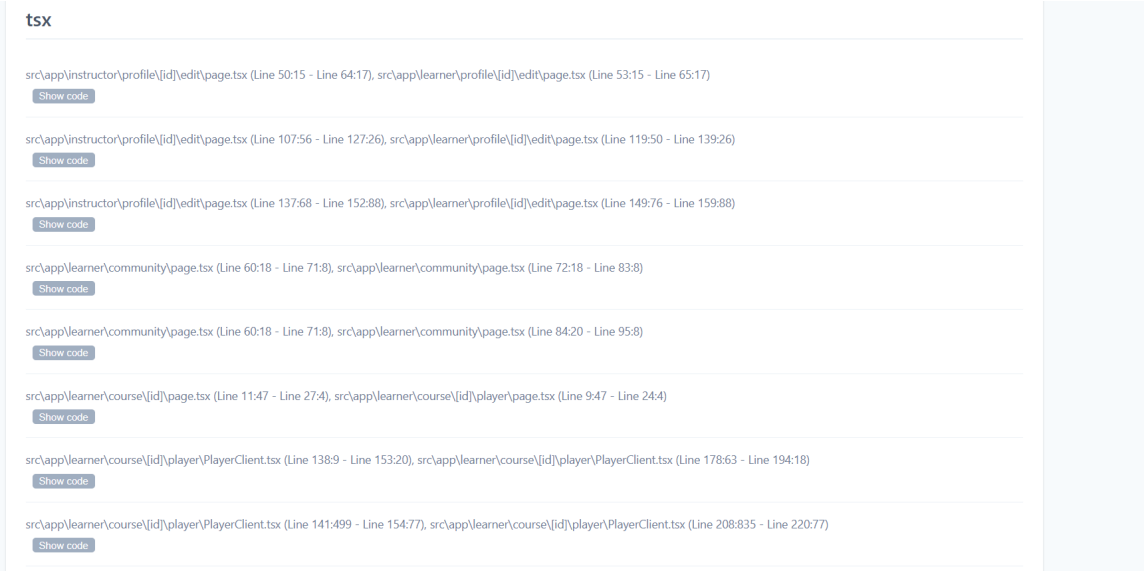


Figure 4: First part of the detected TSX clone locations



Figure 5: Additional detected TSX clone locations

5 Analysis of Some Selected Code Clones

5.1 Profile Input Handling Clone

A clone was detected between:

- `src/app/instructor/profile/[id]/edit/page.tsx`, lines 50–64
- `src/app/learner/profile/[id]/edit/page.tsx`, lines 53–65

The duplicated block contains common form-handling logic, including the `handleInputChange` function. The function reads the input name and value and updates the form state. The

two blocks are almost identical, so this is a **Type 1 clone**.

This duplication may create inconsistency if validation or input-processing behavior is changed in one profile page but not in the other. The common logic should be moved to a shared profile-form component or a reusable hook.



Figure 6: Duplicated input-handling logic in instructor and learner profile pages

5.2 Community Like and Save Logic

A clone was detected inside `src/app/learner/community/page.tsx` between lines 60–71 and lines 72–83. The code updates a JavaScript `Set` by removing an ID when it already exists and adding it otherwise. Similar logic is used for different community actions such as liking and saving a post.

The structure is the same, but the function and state names are different. Therefore, it can be considered a **Type 2 clone**. Keeping two copies means that any change to the toggle behavior must be repeated. A generic helper such as `toggleIdInSet` or a small custom hook can remove this duplication.

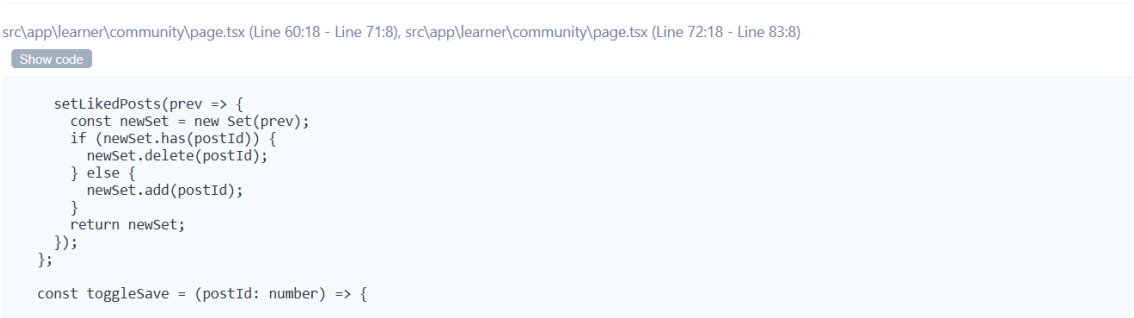


Figure 7: Repeated state-toggle logic in the learner community page

5.3 Resource Download Function

A clone was detected between:

- `src/app/learner/course/[id]/player/PlayerClient.tsx`, lines 178–195
- `src/components/course/CourseModules.tsx`, lines 250–265

Both blocks contain the `downloadResource` function. The function converts Base64 data into bytes, creates a PDF blob, generates a temporary URL, and prepares the file for download. Since the same implementation appears in both files, it is a **Type 1 clone**.

This is an important maintenance issue because changes to file naming, PDF generation, or browser cleanup would have to be made in two places. The function should be moved to a shared utility file and imported by both components.



Figure 8: Duplicated resource-download function

5.4 Other Detected Patterns

The report also found repeated structures in the following areas:

- Course details and course-player pages
- `PlayerClient` and `CourseModules` components
- Privacy Policy and Terms and Conditions pages
- Category and course statistics cards
- Course tables and instructor-application lists

These clones mainly represent repeated page layouts, data cards, tables, and course-module interfaces. Although some repeated layout is understandable, shared components would make future changes easier and more consistent.

6 Proposed Solutions

The following improvements are recommended:

1. Create a shared profile form or profile-edit hook for learner and instructor profile

pages.

2. Replace repeated like/save logic with one generic set-toggle helper or custom hook.
3. Move `downloadResource` to a shared utility module.
4. Create reusable components for legal-document pages, statistics cards, tables, and course-module lists.
5. Pass labels, data, and callback functions through component properties instead of copying complete TSX blocks.
6. Run jscpd again after refactoring to confirm that the duplication percentage has decreased.

No source code was modified during this lab. The detected clones were analyzed, and suitable refactoring solutions were proposed as required by the task.

7 Conclusion

The SkillScribe project was successfully analyzed using jscpd. The tool examined 252 files and detected 55 clones, with 789 duplicated lines and an overall duplication rate of 2.30%. Most clones were found in TSX files because the application contains many similar pages and interface components.

The selected examples showed repeated profile-form handling, community state toggling, and resource-download logic. Moving these blocks into shared hooks, utility functions, and reusable components would reduce repeated maintenance work and help keep the behavior of related pages consistent.